

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

PENGANTAR MIKROKONTROLER

Umar Abdurrahman - 5024241010

2026

1 Langkah-Langkah Percobaan

Terdapat 5 percobaan yang membutuhkan board STM32 Blackpill, ST-Link V2, ESP32 DevKit, kabel USB Type-C, breadboard, LED, resistor, pushbutton, LDR, serta laptop yang telah terpasang VSCode dan terkonfigurasi PlatformIO-nya.

Percobaan pertama yaitu menghubungkan STM32 ke laptop dan mengetes blinking-nya:

1. Mengatur konfigurasi project PlatformIO yang dibuat pada file platformio.ini untuk membaca board STM32 Blackpill dengan framework Arduino.
2. Menghubungkan ST-Link V2 ke STM32 Blackpill melalui pin SWDIO, SWCLK, GND, dan 3.3V sesuai Gambar 2.
3. Menuliskan program Blink dari modul dan melakukan proses Build serta Upload melalui PlatformIO, seperti pada Gambar 3.
4. Mengamati LED onboard PC13 untuk memastikan LED berkedip secara berulang, seperti pada Gambar 4.

Percobaan kedua yaitu menghubungkan ESP32 ke laptop dan mengetes blinking-nya:

1. Mengubah konfigurasi PlatformIO untuk membaca board ESP32 DevKit dengan framework ESP-IDF.
2. Menghubungkan ESP32 ke laptop menggunakan kabel USB Type-C.
3. Mengganti program "main.cpp" dengan program "main.c" pada modul.
4. melakukan proses Build serta Upload melalui PlatformIO.
5. Mengamati LED onboard ESP32 untuk memastikan program berjalan dengan baik, seperti pada Gambar 5.

Percobaan ketiga yaitu mengamati rangkain Pull-up dan Pull-down resistor external maupun internal.

1. Mengubah konfigurasi PlatformIO menjadi menggunakan framework Arduino dan memasukkan program "main.cpp" dari modul.
2. Merangkai dua pushbutton yang terhubung pada GPIO 32 dan GPIO 33 menggunakan resistor pull-up dan pull-down eksternal sesuai Gambar 6.
3. Melakukan Build dan Upload seperti pada Gambar 7.
4. Menekan kedua tombol secara bergantian dan mengamati kondisi logika melalui Serial Monitor seperti pada Gambar 8 dan Gambar 9.
5. Melepas kedua resistor eksternal tanpa mengubah posisi pushbutton, kemudian mengaktifkan konfigurasi pull-up dan pull-down internal pada program.
6. Melakukan Build dan Upload kembali, kemudian menguji tombol pull-up dan pull-down secara terpisah seperti pada Gambar 10, Gambar 11, dan Gambar 12.

Percobaan keempat yaitu terkait menggunakan debouncing pada program agar menghindari membaca noise penekanan tombol.

1. Merangkai pushbutton yang terhubung pada GPIO 25 ESP32 dengan konfigurasi pull-up internal (`INPUT_PULLUP`) sesuai skema rangkaian pada modul.
2. Menuliskan program yang ada pada modul.
3. Mengatur nilai `DEBOUNCE_DELAY` sebesar 50 ms, kemudian melakukan Build dan Upload program.
4. Menekan tombol satu kali dan membandingkan perubahan kedua penghitung melalui Serial Monitor seperti pada Gambar 14 .
5. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian melakukan Build dan Upload kembali serta mengamati hasil pengujian seperti pada Gambar 13.
6. Mengulangi pengujian menggunakan nilai 200 ms seperti pada Gambar 15.

Percobaan kelima yaitu melihat hasil pembacaan ADC dari input LDR pada mikrokontroler.

1. Merangkai LDR dan resistor $10k\Omega$ sebagai pembagi tegangan, dengan titik tengah rangkaian terhubung ke GPIO 34 ESP32.
2. Menuliskan program untuk membaca nilai ADC seperti pada modul.
3. Melakukan Build dan Upload program, kemudian membuka Serial Monitor.
4. Mengamati nilai ADC dan tegangan ketika LDR dalam kondisi minim cahaya (ditutup tangan), seperti pada Gambar 16.
5. Memberikan pencahayaan sedang dan tinggi pada LDR, kemudian mengamati perubahan nilai pembacaan seperti pada Gambar 17 dan Gambar 18.
6. Membandingkan hasil pembacaan ADC berdasarkan perubahan intensitas cahaya.

2 Analisis Hasil Percobaan

Percobaan pertama terkait LED Blink pada STM32 berhasil dilakukan. LED dapat berkedip setelah program di-build dan upload yang terhubung melalui ST-Link V2. Hal ini menunjukkan bahwa konfigurasi PlatformIO, koneksi programmer melalui protokol SWD, dan eksekusi program pada board STM32 berjalan dengan baik.

Percobaan kedua terkait LED Blink pada ESP32 berhasil dilakukan. LED dapat berkedip setelah program diunggah melalui koneksi USB. Hasil ini menunjukkan bahwa ESP32 berhasil diprogram menggunakan framework ESP-IDF melalui PlatformIO. Berbeda dengan framework Arduino, program ESP-IDF menggunakan fungsi `app_main()` sebagai titik masuk program.

Pengujian pull-up dan pull-down menggunakan resistor eksternal maupun internal pada percobaan ketiga berhasil dilakukan. Kedua konfigurasi dapat memberikan kondisi logika pada pin input sesuai dengan rangkaian dan program yang digunakan. Dari hasil pengamatan, konfigurasi pull-up

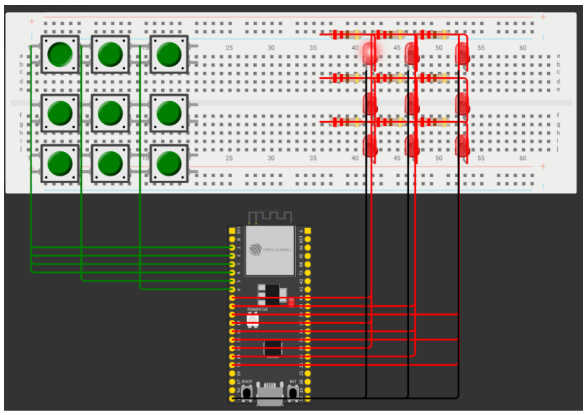
menghasilkan kondisi HIGH ketika tombol tidak ditekan dan LOW ketika ditekan. Sebaliknya, pull-down menghasilkan kondisi LOW ketika tombol tidak ditekan dan HIGH ketika ditekan, yang mana sudah sesuai denga teori. Dengan demikian, kedua konfigurasi memiliki kondisi logika yang berlawanan, tetapi sama-sama dapat digunakan untuk menghindari kondisi floating pada input GPIO. Pada percobaan ini juga terdapat kendala, dimana pengamatan Serial Monitor terdapat jeda pembaruan tampilan. Hal ini perlu diperhatikan ketika mengamati perubahan kondisi tombol karena program melakukan pembacaan dan pencetakan data secara berkala.

Pada percobaan keempat terkait debouncing, pengujian menunjukkan bahwa nilai `DEBOUNCE_DELAY` memengaruhi proses penyaringan perubahan sinyal pada tombol. Tanpa debouncing, perubahan sinyal akibat bouncing berpotensi menyebabkan satu kali penekanan tombol terbaca sebagai beberapa kali input. Nilai `DEBOUNCE_DELAY` yang terlalu kecil berpotensi belum menyaring seluruh perubahan sinyal tersebut. Sebaliknya, waktu debounce yang terlalu besar dapat membuat respons tombol menjadi lebih lambat (ada tekanan yang tak terbaca ketika ditekan cepat). Oleh karena itu, diperlukan pemilihan waktu debounce yang sesuai agar pembacaan tombol lebih stabil tanpa mengurangi responsivitas secara berlebihan.

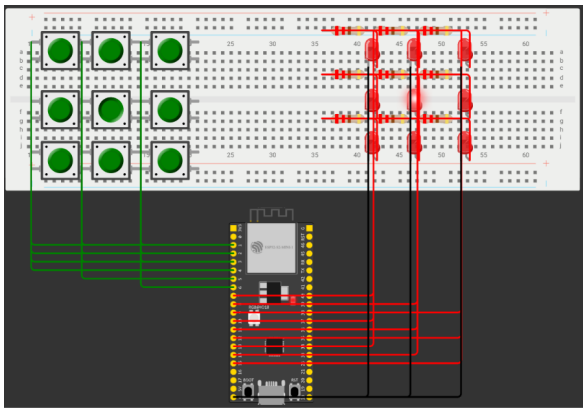
Hasil pengujian percobaan kelima menunjukkan bahwa ketika intensitas cahaya berkurang, nilai tegangan keluaran rangkaian pembagi tegangan semakin mendekati 0 V. Sebaliknya, ketika LDR mendapatkan pencahayaan lebih tinggi, nilai tegangan keluarannya meningkat. Perubahan tersebut terjadi karena resistansi LDR dipengaruhi oleh intensitas cahaya yang diterimanya. Perubahan resistansi ini memengaruhi tegangan pada titik tengah pembagi tegangan yang kemudian dibaca oleh ADC ESP32. Dengan demikian, ADC dapat digunakan untuk membaca perubahan kondisi lingkungan dalam bentuk nilai digital (sinyal kontinu yang di-sampling dan dikuantisasi), bukan hanya kondisi HIGH dan LOW seperti pada input digital.

3 Hasil Tugas Modul

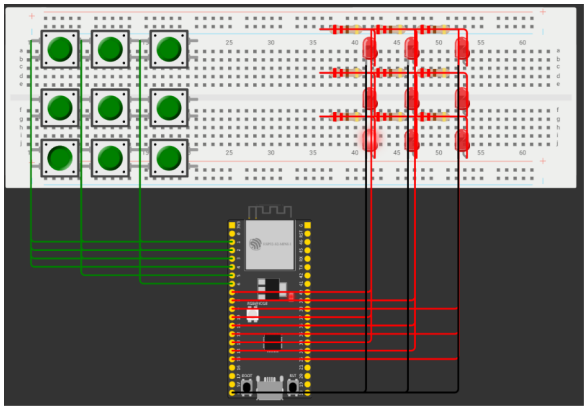
Hasil simulasi Matrix Pushbutton 3x3 yang mengendalikan Matrix LED 3x3 pada titik (1,1), (2,2), dan (3,1):



(a) Tombol (1,1) menyalakan LED (1,1)



(b) Tombol (2,2) menyalakan LED (2,2)



(c) Tombol (3,1) menyalakan LED (3,1)

Gambar 1: Hasil simulasi

Listing 1: Sumber kode untuk menjalankan simulasi

```
// Definisi Pin GPIO untuk Tombol Matrix 3x3 pada ESP32-S2
const int rowPins[3] = {1, 2, 3}; // Baris tombol (Output)
const int colPins[3] = {4, 5, 6}; // Kolom tombol (Input Pullup)

// Definisi Pin GPIO untuk LED Matrix 3x3
const int ledPins[3][3] = {
  {7,  8,  9},  // Baris 0 (LED 1,1 | 1,2 | 1,3)
  {10, 11, 12}, // Baris 1 (LED 2,1 | 2,2 | 2,3)
  {13, 14, 15}  // Baris 2 (LED 3,1 | 3,2 | 3,3)
};

// State dan variabel debouncing
bool ledState[3][3] = { {false, false, false},
                        {false, false, false},
                        {false, false, false} };

bool lastButtonState[3][3] = { {HIGH, HIGH, HIGH},
                               {HIGH, HIGH, HIGH},
                               {HIGH, HIGH, HIGH} };
```

```
20
21 bool buttonState[3][3] = { {HIGH, HIGH, HIGH},
22                           {HIGH, HIGH, HIGH},
23                           {HIGH, HIGH, HIGH} };
24
25 unsigned long lastDebounceTime[3][3] = { {0, 0, 0},
26                                          {0, 0, 0},
27                                          {0, 0, 0} };
28
29 const unsigned long debounceDelay = 50;
30
31 void setup() {
32   // Inisialisasi pin baris tombol sebagai OUTPUT
33   for (int r = 0; r < 3; r++) {
34     pinMode(rowPins[r], OUTPUT);
35     digitalWrite(rowPins[r], HIGH);
36   }
37
38   // Inisialisasi pin kolom tombol sebagai INPUT_PULLUP
39   for (int c = 0; c < 3; c++) {
40     pinMode(colPins[c], INPUT_PULLUP);
41   }
42
43   // Inisialisasi pin LED matrix sebagai OUTPUT
44   for (int r = 0; r < 3; r++) {
45     for (int c = 0; c < 3; c++) {
46       pinMode(ledPins[r][c], OUTPUT);
47       digitalWrite(ledPins[r][c], LOW);
48     }
49   }
50 }
51
52 void loop() {
53   // Pemindaian Matrix Baris (Row-Scanning)
54   for (int r = 0; r < 3; r++) {
55     digitalWrite(rowPins[r], LOW); // Aktifkan baris aktif secara bergantian
56
57     for (int c = 0; c < 3; c++) {
58       int reading = digitalRead(colPins[c]); // Membaca status tombol (LOW saat
     ditekan)
59
60       // Deteksi bouncing
61       if (reading != lastButtonState[r][c]) {
62         lastDebounceTime[r][c] = millis();
63       }
64
65       // Pastikan pembacaan stabil setelah melewati debounceDelay
66       if ((millis() - lastDebounceTime[r][c]) > debounceDelay) {
67         if (reading != buttonState[r][c]) {
68           buttonState[r][c] = reading;
69         }
70       }
71
72       lastButtonState[r][c] = reading;
```

```
73
74        if (buttonState[r][c] == LOW) {
75          digitalWrite(ledPins[r][c], HIGH);
76        } else {
77          digitalWrite(ledPins[r][c], LOW);
78        }
79      }
80
81      digitalWrite(rowPins[r], HIGH); // Matikan kembali baris
82      delay(2); // Jeda singkat stabilisasi antar baris
83    }
84  }
```

1. Debouncing bukan menjadikan tekanan di dua tombol atau lebih dianggap menekan satu tombol yang lebih awal tertekakan, melainkan memfilter anggapan/derau banyak tekanan pada satu tekanan tombol. Bouncing dapat terjadi karena saat kontak logam di dalam pushbutton saling bersentuhan atau terlepas, pegas logam mengalami getaran mikroskopis selama beberapa milidetik. Akibatnya, sinyal listrik berosilasi naik-turun dengan cepat. Pada desain rangkaian matriks sendiri, mikrokontroler memindai baris secara berulang dengan sangat cepat, osilasi sinyal mentah ini akan terbaca sebagai banyak event penekanan terpisah dalam satu kali tekanan fisik. Tanpa debouncing sistem dapat membaca satu kali tekan sebagai beberapa kali perubahan status.

2. Rangkaian pada simulasi menggunakan konfigurasi Internal Pull-Up pada pin kolom tombol sehingga HI ketika tidak ditekan dan LOW ketika ditekan. Rangkaian tidak boleh dibiarkan Floating karena bisa membuatnya menyerap derau elektromagnetik dari lingkungan sekitar sehingga mikrokontroler akan membaca nilai HIGH dan LOW secara acak meskipun tidak ada tombol yang disentuh, menyebabkan sistem mendeteksi input palsu.

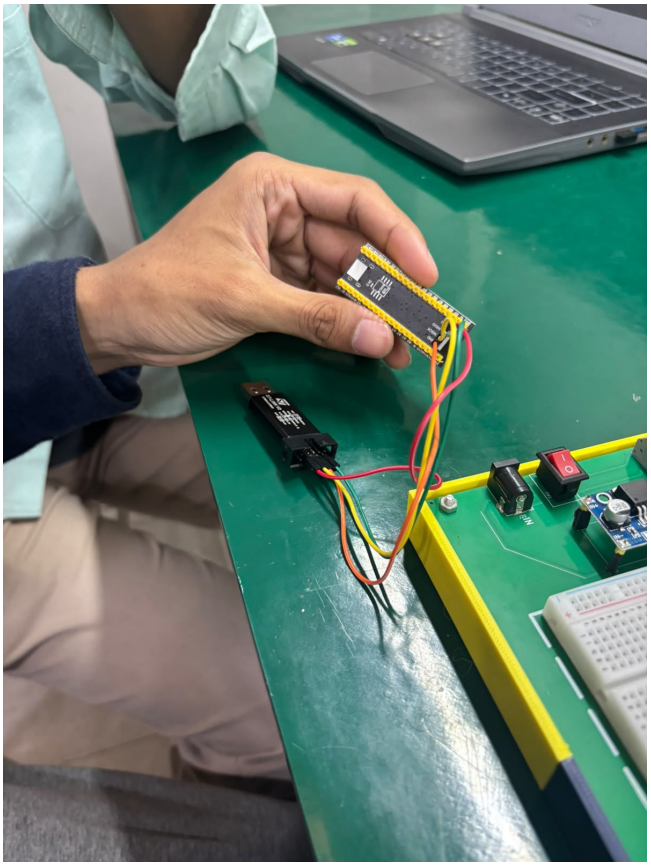
3. Input digital hanya mengenal HI dan LOW, yang mana ditentukan oleh threshold yang diatur. Sementara input analog dapat mengukur spektrum tegangan kontinu dan mengkuantisasinya menjadi nilai numerik dalam rentang bit yang dimiliki. Karena LDR dapat membuat lebih banyak nilai, LED dapat menyala dari gelap ke terang karena menerima input tegangan yang lebih kotinu.

4 Kesimpulan

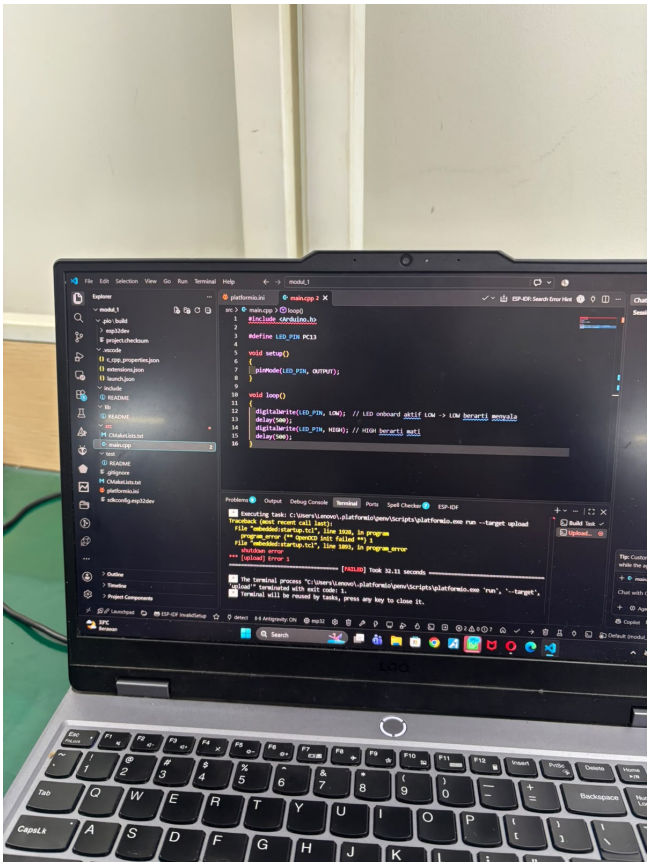
Dari hasil percobaan-percobaan yang didapat, disimpulkan bahwa board STM32 Blackpill dan ESP32 berhasil menjalankan program Blink menggunakan framework Arduino dan ESP-IDF. Pengujian pull-up dan pull-down menunjukkan bahwa konfigurasi resistor eksternal maupun internal dapat digunakan untuk menentukan kondisi logika input tombol dan menghindari kondisi floating. Percobaan debouncing dapat dipengaruhi oleh perubahan nilai `DEBOUNCE_DELAY`. Percobaan ADC menggunakan LDR menunjukkan bahwa perubahan intensitas cahaya memengaruhi tegangan keluaran rangkaian pembagi tegangan yang dibaca oleh ADC ESP32.

5 Lampiran

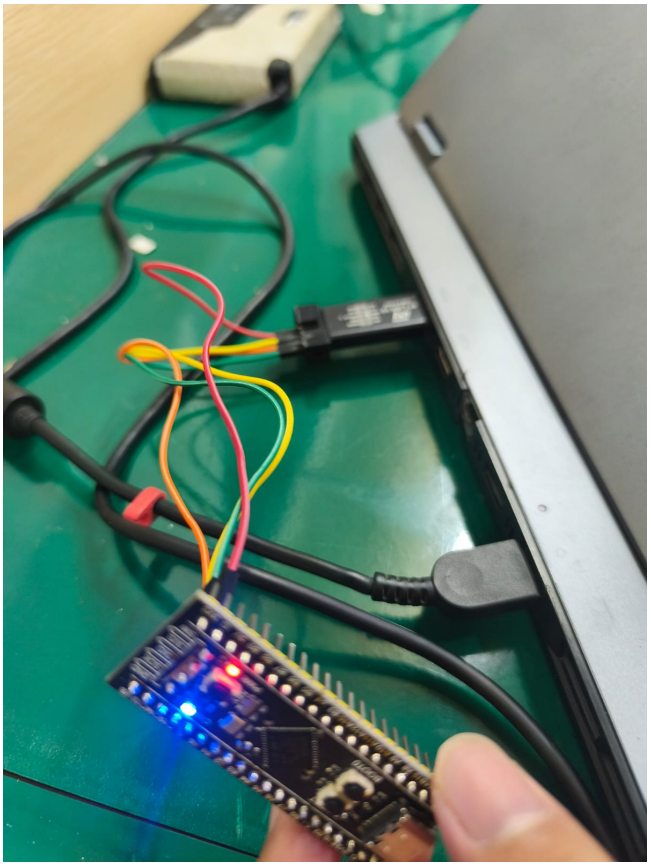
5.1 Dokumentasi saat praktikum



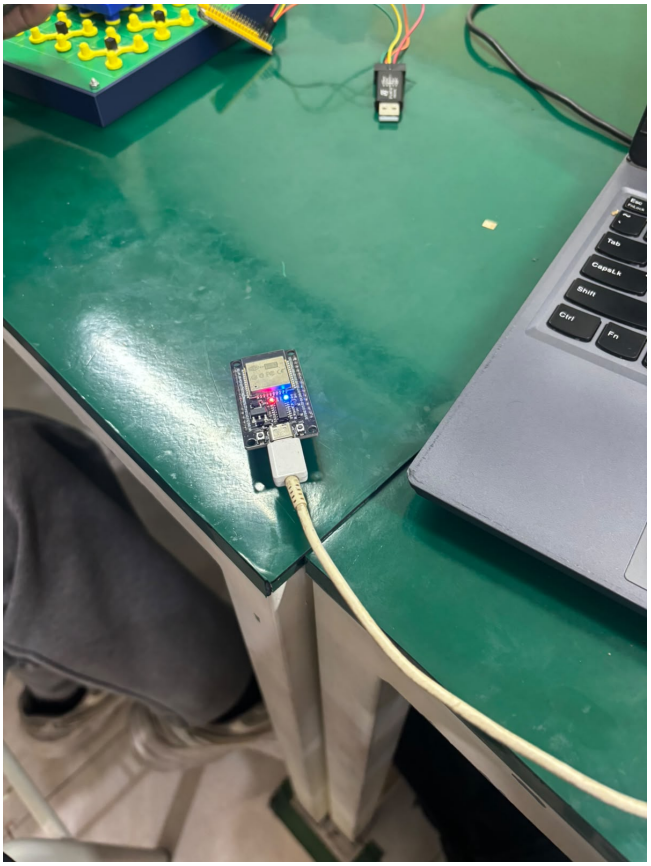
Gambar 2: Koneksi ST-Link V2 dengan STM32 Blackpill



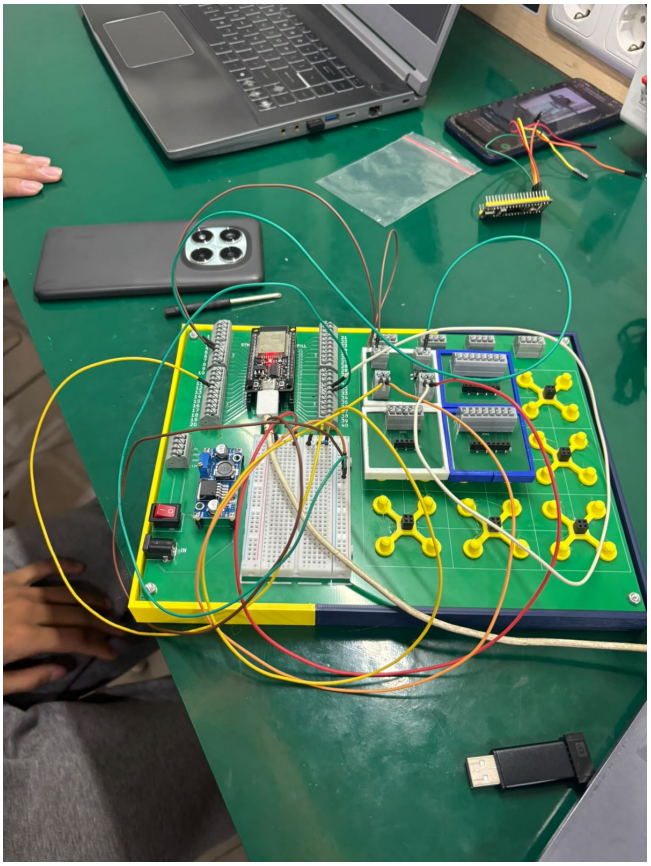
Gambar 3: Program Blink pada STM32 Blackpill



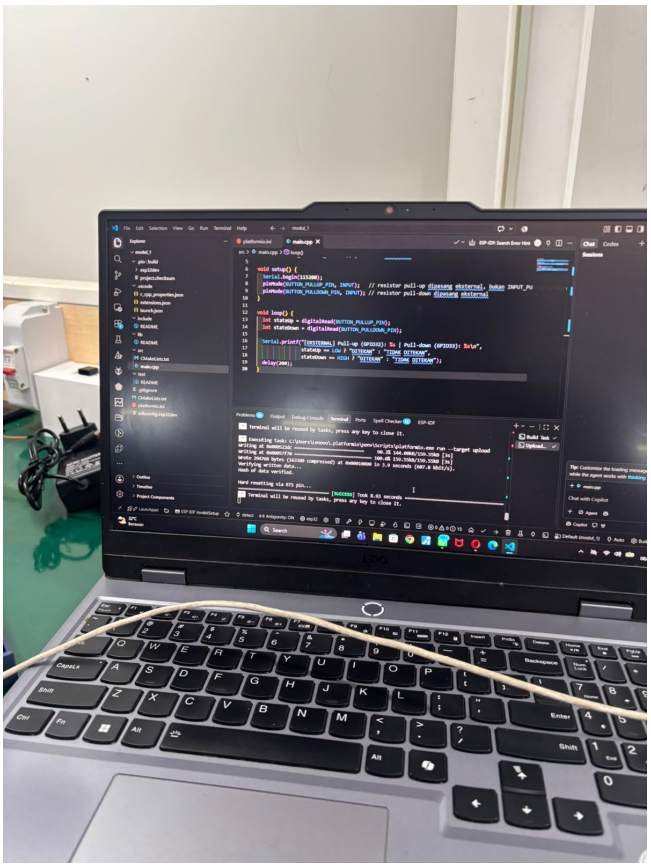
Gambar 4: Hasil percobaan Blink pada STM32 Blackpill



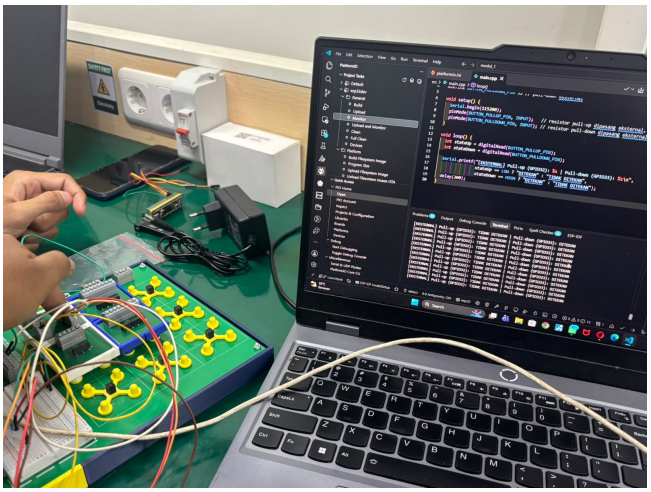
Gambar 5: Hasil percobaan Blink pada ESP32



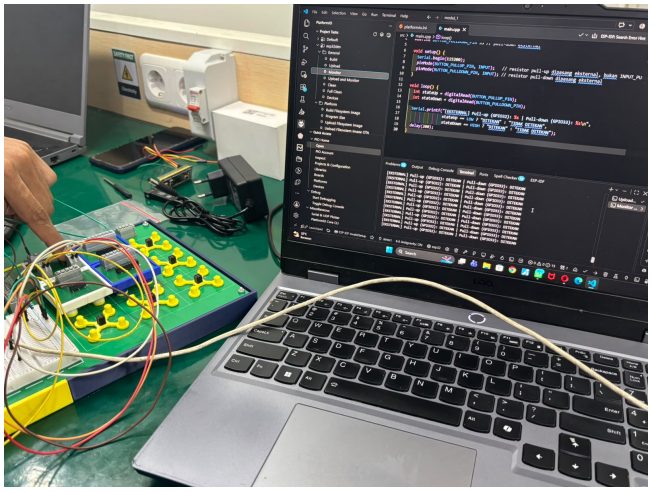
Gambar 6: Rangkaian pull-up dan pull-down menggunakan resistor eksternal



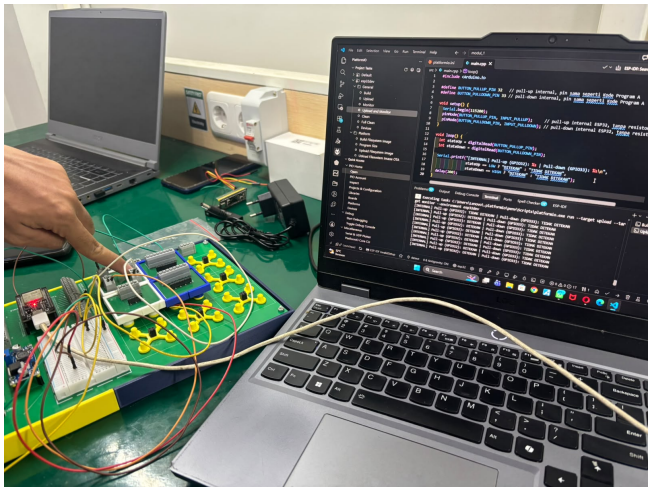
Gambar 7: Program pengujian pull-up dan pull-down eksternal



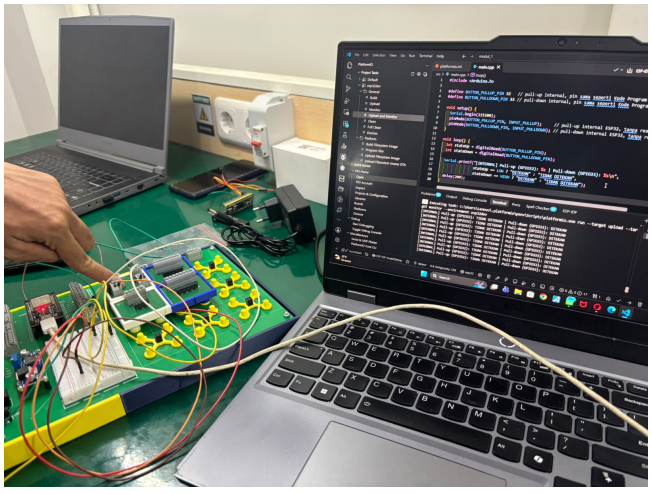
Gambar 8: Hasil pengujian pull-up eksternal



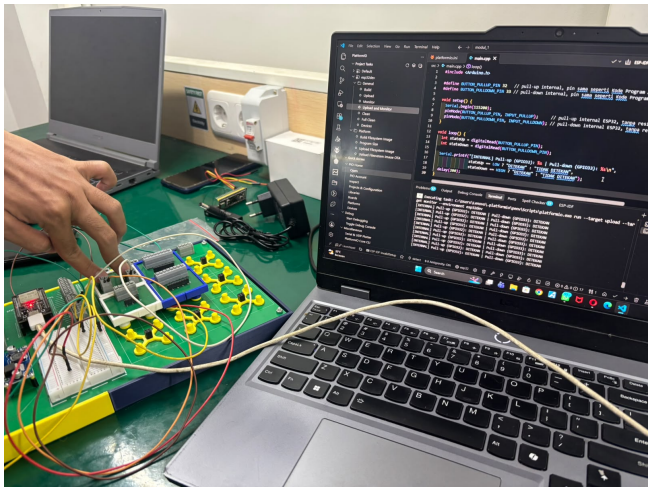
Gambar 9: Hasil pengujian pull-down eksternal



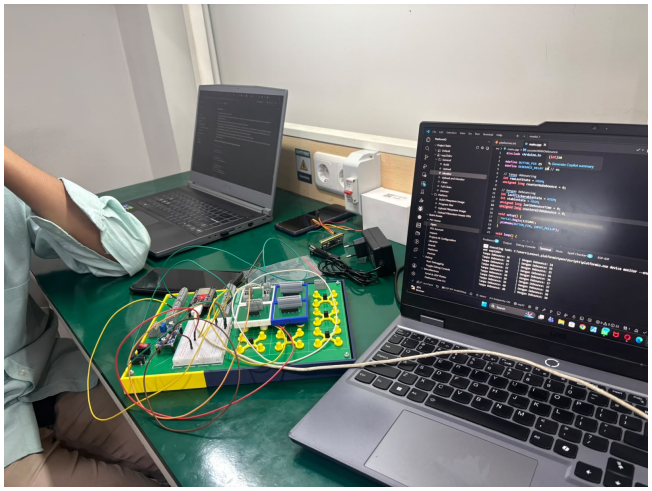
Gambar 10: Hasil pengujian pull-up internal



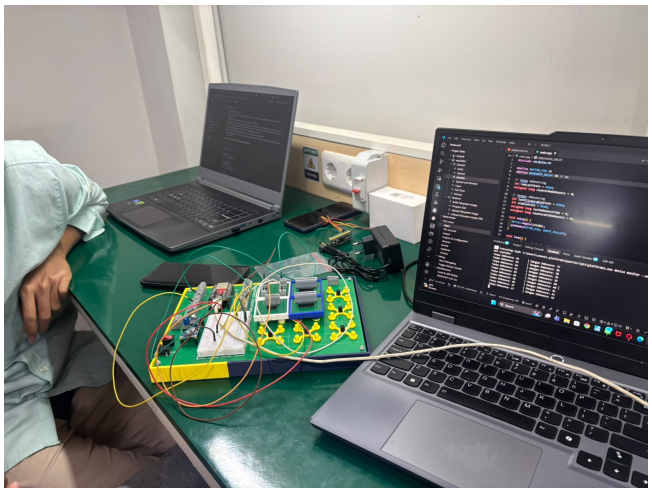
Gambar 11: Hasil pengujian pull-down internal



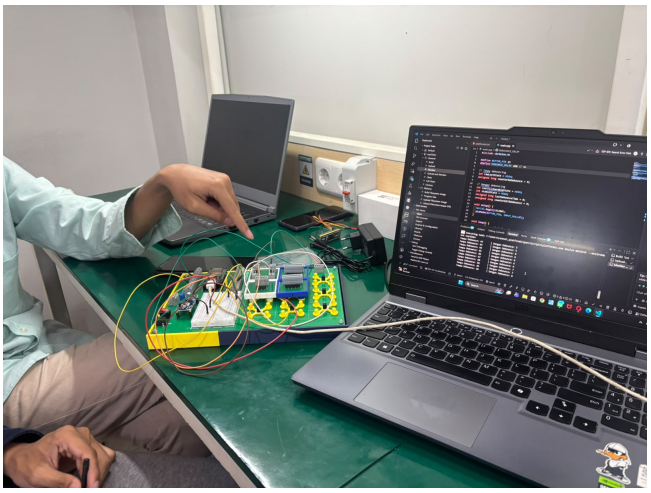
Gambar 12: Hasil pengujian pull-up dan pull-down internal ketika kedua tombol ditekan



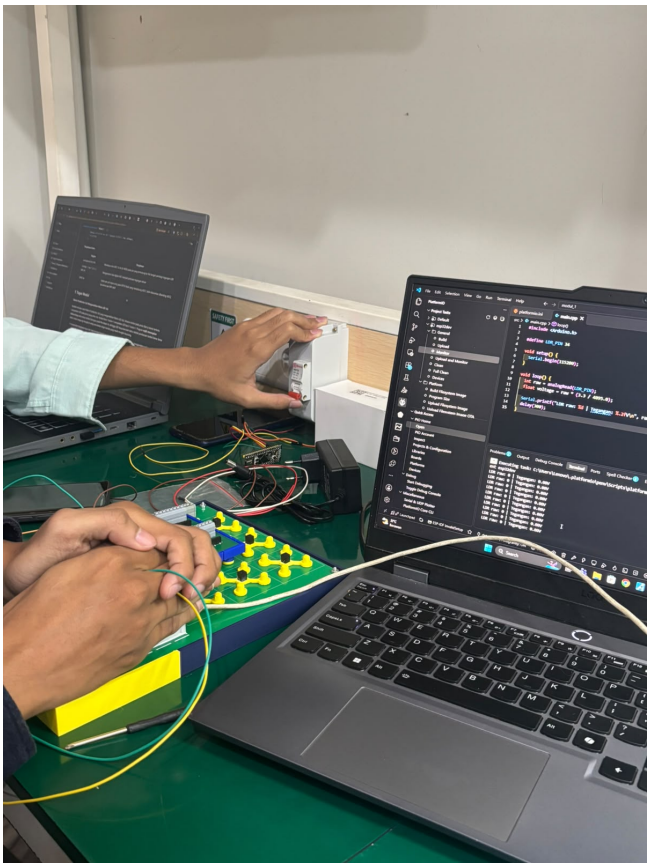
Gambar 13: Hasil pengujian debouncing dengan delay 10 ms



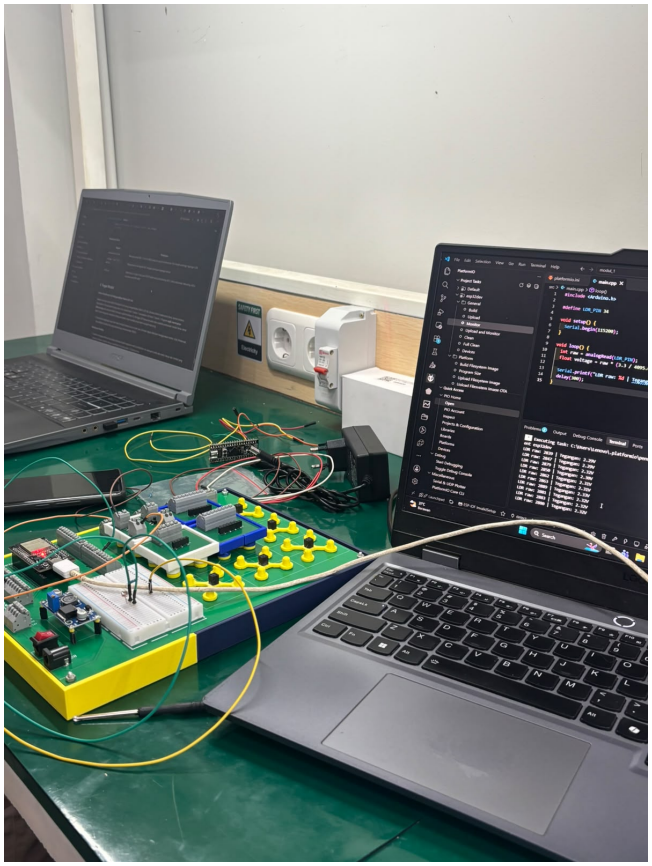
Gambar 14: Hasil pengujian debouncing dengan delay 50 ms



Gambar 15: Hasil pengujian debouncing dengan delay 200 ms



Gambar 16: Hasil pembacaan ADC pada kondisi pencahayaan rendah



Gambar 17: Hasil pembacaan ADC pada kondisi pencahayaan sedang



Gambar 18: Hasil pembacaan ADC pada kondisi pencahayaan tinggi